

# **DTS103TC**

# **Data Structure and Algorithms**

## **Lecture 5: Data Structures 3**

Dr. Qi Chen  
School of AI and Advanced Computing

# Learning outcome

- Hash table

# Hash Table

- Another kind of Table
- Use an array named  $T$  of capacity  $m$
- Define a **hash function** that returns an integer  $h(k)$ 
  - Must return an integer between 0 and  $m-1$
  - $h()$  must always return the same integer for a given key
- Store the key and info at  $T[h(k)]$
- $O(1)$  on **average** for insertion, lookup, and deletion

# Hash Table

- A hash table stores data in key-value pairs:
  - **Key** - used to compute the index (via a hash function)
  - **Value** - the data associated with the key

| Key | Value |
|-----|-------|
|-----|-------|

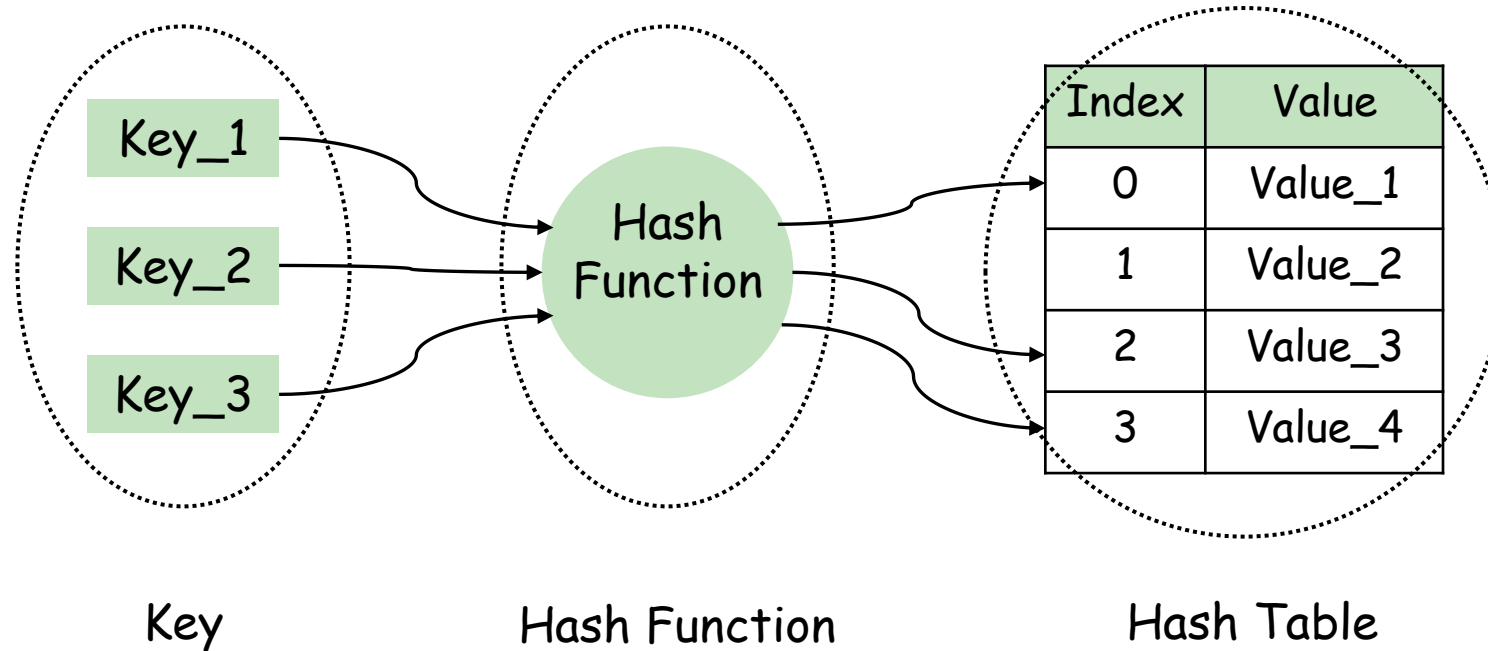
- Example: ("student\_id", "Alice")
- Keys must be unique, but values can be duplicated.

# What is a Hash function?

- A **hash function** is a well-defined function that maps a key to an index in a hash table.
- A hash function  **$h$** :
  - Takes a key  **$k$**  as input
  - Returns an index  **$h(k)$** , which is called the **hash value** or simply **hashes**
- **$h$**  must return an integer between 0 and  $m-1$
- **$h$**  must always return the same integer for a given key

# Hash Function

- Let  $k$  be a key and  $h(k)$  be a hash function.
- The function  $h(k)$  computes an index where the data associated with  $k$  will be stored.



# Hash Function

The division method:

$$h(k) = k \bmod m \text{ (or } k \% m \text{)}$$

- Example:
  - $k = 1234, m = 11$
  - $h(k) = 1234 \bmod 11 = 2$
- For Strings: Convert characters to numbers (e.g., ASCII)
  - $k = \text{"test"} \rightarrow 74, 65, 73, 74$
  - $h(k) = (74+65+73+74) \bmod 11 = 0$
- Choosing  $m$  (often a prime number) helps reduce collisions

# Hash Function

- Multiplication Method

$$h(k) = \lfloor m (kA \bmod 1) \rfloor$$

$A$  is a constant ( $0 < A < 1$ )

- Steps:

- 1. Compute  $kA$
- 2. Take the fractional part
- 3. Multiply by  $m$  and take the floor

- We use the fractional part of  $kA$  to generate a well-distributed index



# Why Hashing?

- When storing a large amount of data, we need efficient operations.
  - Linear search:  $O(n)$
  - Binary search (on sorted data):  $O(\log n)$
  - Hashing:  $O(1)$  on average for insertion, lookup, and deletion

# Hash table give $O(1)$ performance?

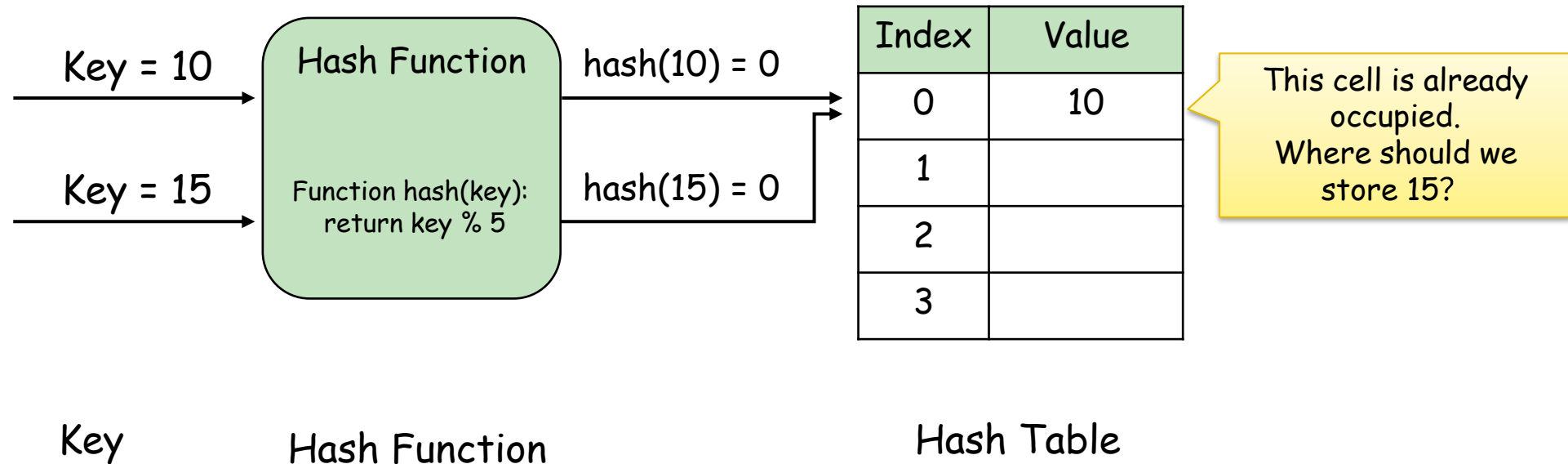
- When inserting, searching, or deleting a key:
  - Compute the hash value  $h(k)$
  - This gives the exact index in the array
  - Access the element directly
- → No need to search. Therefore, operations take  $O(1)$  time (on average)
- What if the position is already occupied?
  - A hash function does not guarantee unique outputs.
  - Different keys may produce the same hash value.
  - This is called a collision.

# Hash Function

- Characteristics of a Good Hash Function
  - Minimizes collisions
  - Distributes keys evenly across the table
  - Efficient to compute  $O(1)$
- A good hash function reduces clustering and improves performance
  - Bad hash function → many collisions → slow performance
  - Good hash function → fewer collisions → near  $O(1)$

# Hash Collision

- When the hash function generates the same index for multiple keys, there will be a conflict (which value should be stored at that index?). This is called a **hash collision**.



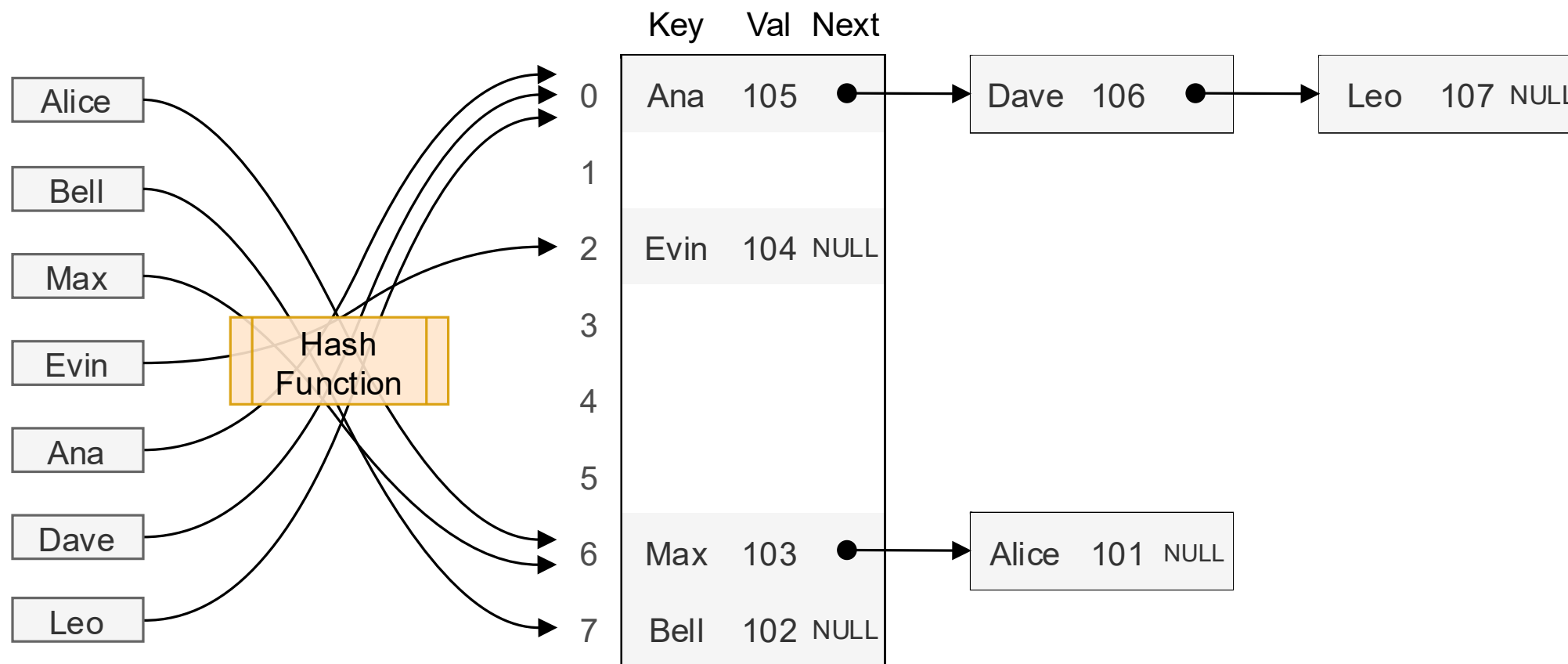
# Hash Collision

- When the hash function generates the same index for multiple keys, there will be a conflict (which value should be stored at that index?). This is called a **hash collision**.
- We can resolve hash collisions using the following methods:
  - Chaining
  - Open Addressing: Linear/Quadratic Probing and Double Hashing

# Chaining

- If multiple keys map to the same index, they are stored in a linked list at that index.
  - Each index can store multiple elements.
  - Each slot  $j$  contains a pointer to the head of the linked list.
  - If no element is stored at index  $j$ , then  $j = \text{NULL}$
- Example:  
Index 2  $\rightarrow (k1, v1) \rightarrow (k2, v2) \rightarrow (k3, v3)$

# Chaining



# Open Addressing

- Unlike separate chaining, open addressing does not store multiple elements in the same slot.
  - Each slot contains either:
    - one key
    - or is empty (NULL)
- When a collision occurs, we search for another empty slot in the table.
- Different techniques used in open addressing:
  - Linear probing
  - Quadratic probing
  - Double hashing



# Linear Probing

- Collision is resolved by checking the next available slot.

$$h(k, i) = (h_1(k) + i) \bmod m$$

where

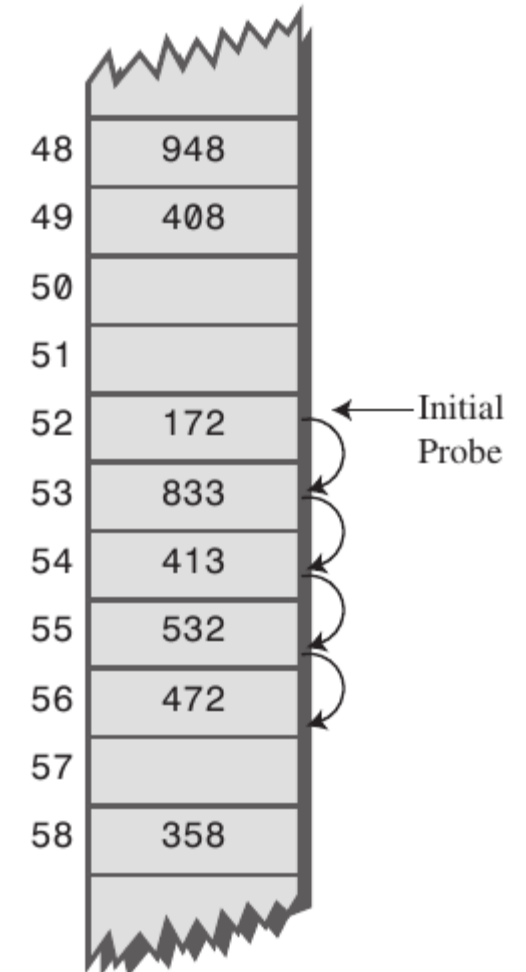
- $i = \{0, 1, 2, \dots\}$
  - $h_1(k) = k \bmod m$
- If a collision occurs at  $h(k, 0)$ , then  $h(k, 1)$  is checked.  
Increment hash value by a constant 1, until free slot is found.

# Linear Probing

- Example:
  - Insert  $k_1 \rightarrow$  index 3
  - Insert  $k_2 \rightarrow$  index 3 (collision)  $\rightarrow$  move to index 4
  - Insert  $k_3 \rightarrow$  index 3 (collision)  $\rightarrow$  index 4 (collision)  $\rightarrow$  move to index 5
- Advantages:
  - Simple to implement
- Disadvantages:
  - Causes primary clustering
  - Clusters grow and increase collisions

# Primary Clustering

- In linear probing, consecutive occupied slots form clusters.
- New elements are more likely to join these clusters, making them grow larger over time.
- Result:
  - More collisions
  - Longer search time
  - Performance degrades



# Linear Probing

- Let us consider a simple hash function as "key mod 7" and a sequence of keys as 50, 700, 76, 85, 92, 73, 101.

# Quadratic Probing

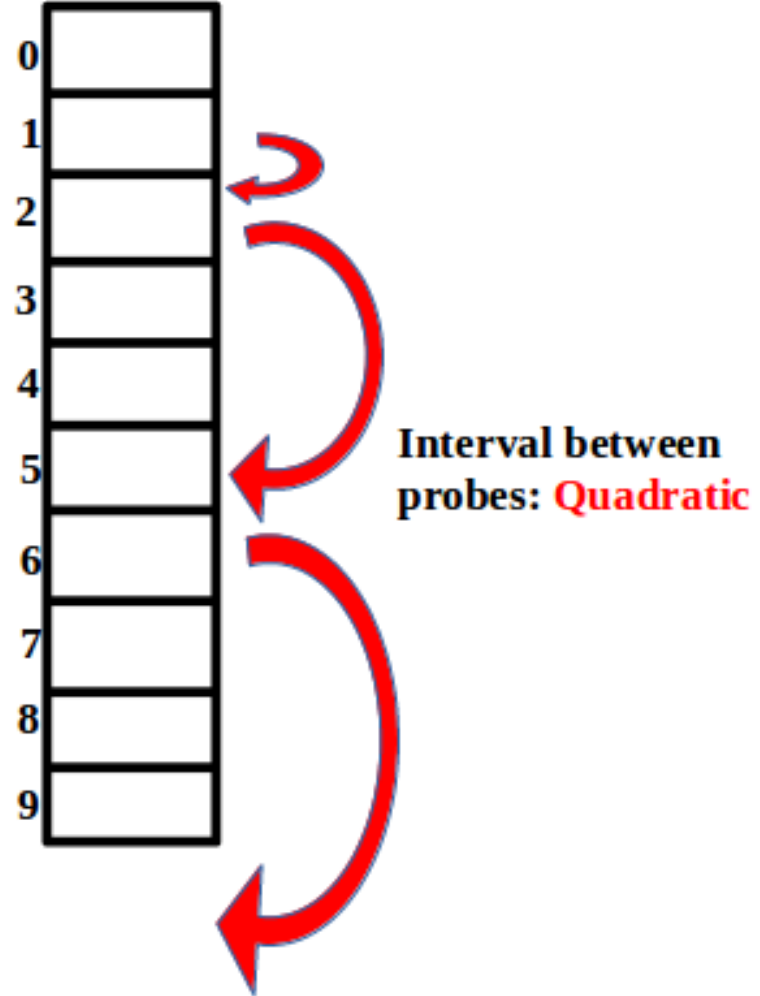
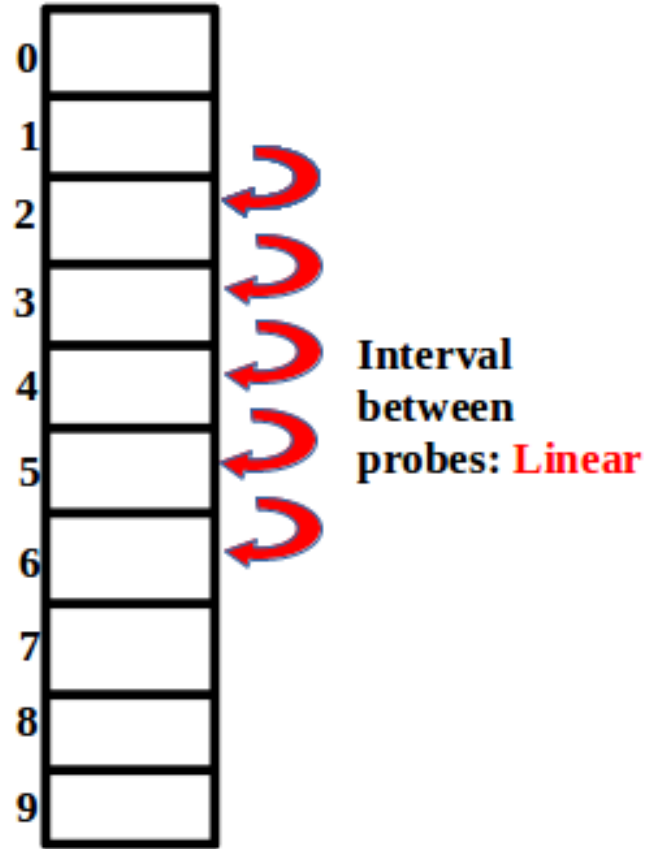
- Similar to linear probing, but the step size increases.

$$h(k, i) = (h_1(k) + c_1 * i + c_2 * i * i) \bmod m$$

where

- $c_1$  and  $c_2$  are positive constants
- $i = \{0, 1, 2, \dots\}$
- Instead of checking consecutive slots, we jump further away each time.
- Reduces primary clustering.
- May still cause secondary clustering.
  - keys with the same initial hash follow the same probing sequence

# Clustering



# Double Hashing

- If a collision occurs, a second hash function is used to determine the step size.

$$h(k,i) = (h_1(k) + i * h_2(k)) \bmod m$$

where:

- $h_1(k)$  = primary hash function
  - $h_2(k)$  = secondary hash function
  - $i = 0, 1, 2, \dots$
- 
- Each key follows a different probing sequence.
  - Avoids clustering

# Table Size

Choosing an appropriate **table size ( $m$ )** is important:

- $m$  is often chosen as a prime number to reduce collisions
- If  $m$  is too large  $\rightarrow$  wasted space
- If  $m$  is too small  $\rightarrow$  more collisions

A good table size balances space usage and collision rate

- Question: What happens as table size approaches 1?



# Dealing with A Full Table

- When the table becomes too full (high load factor  $\alpha = n/m$ ), we resize it:
  1. Allocate a larger hash table
  2. Rehash all existing elements into the new table
  3. Delete the old table
- Example:
  - Old table:  $m = 10 \rightarrow h(k) = k \bmod 10$
  - New table:  $m = 20 \rightarrow h(k) = k \bmod 20$

# Applications

- Some of Hash Table (Function) applications:
  - Message Digest (MD5, SHA256, ...)
  - Password verification
  - String matching (Rabin-Karp algorithm)
  - Dictionaries (e.g., Python dict)
  - Caching and database indexing
  - Etc.
- Key idea:  
Used when fast insertion, lookup and deletion  $O(1)$  are required

# Question from Past Exam Paper

## Question 2. [7+4+7=18 MARKS]

- i. Insert the keys 15, 22, 8, 19, 4, 31, 11 into the hash table using the hash function  $h(k) = k \bmod 7$  and linear probing for collision resolution. Show the contents of the hash table after each insertion. [7 marks]
- ii. Consider searching for the key 8 in the hash table (from question i). Describe the steps you would follow to find the key. [4 marks]
- iii. If key 15 is removed from the hash table (from question i), what happens to the subsequent elements? Describe the steps involved in deleting the key 15 from the hash table that uses linear probing for collision resolution. [7 marks]

# Learning outcome

- Hash table